

ASSESSMENT

NAME: G. Yashmita

REG NO: 192511032

COURSE: CSA0401

Col - AT1 - Short Answer Test



Question 1

- (a) Classify different types of operating systems suitable for this scenario and justify your choices.

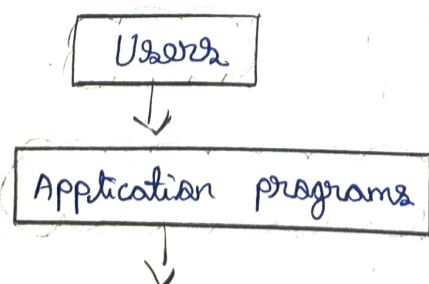
Types:

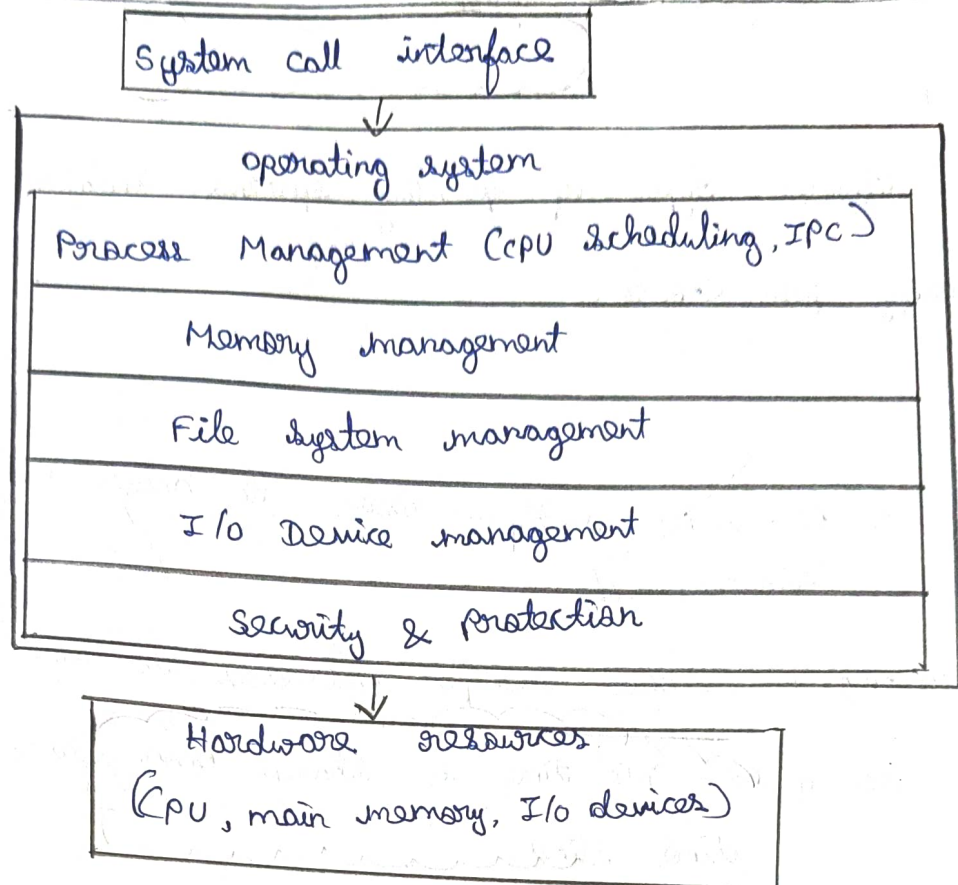
1. Multi-user OS - Allows multiple users to access and use the system simultaneously.
2. Multitasking OS - Allows multiple processes to run concurrently.
3. Time-sharing OS - CPU time is shared among users / processes using time slices.
4. Multiprocessing OS - Supports multiple CPUs / cores for parallel execution.

Justification

A modern OS for multiple users and processes must share CPU, memory and I/O devices efficiently, ensure fairness, protection and high throughput. Therefore, a time-sharing multitasking multi-user OS with multiprocessing capability is most suitable.

- (b) Draw the structure of an operating system showing how resources are managed.



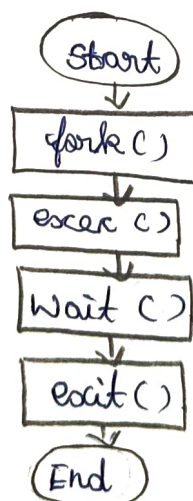


c) Write a sequence of system calls required to create and execute a new process.

System call sequence

- 1) `fork()` - creates a new child process.
- 2) `exec()` - Loads and executes a new program
- 3) `wait()` - Parent waits for the child process to finish.
- 4) `exit()` - child terminates after execution

Flow



Question 2

(a) Categorize system calls used for file and device management

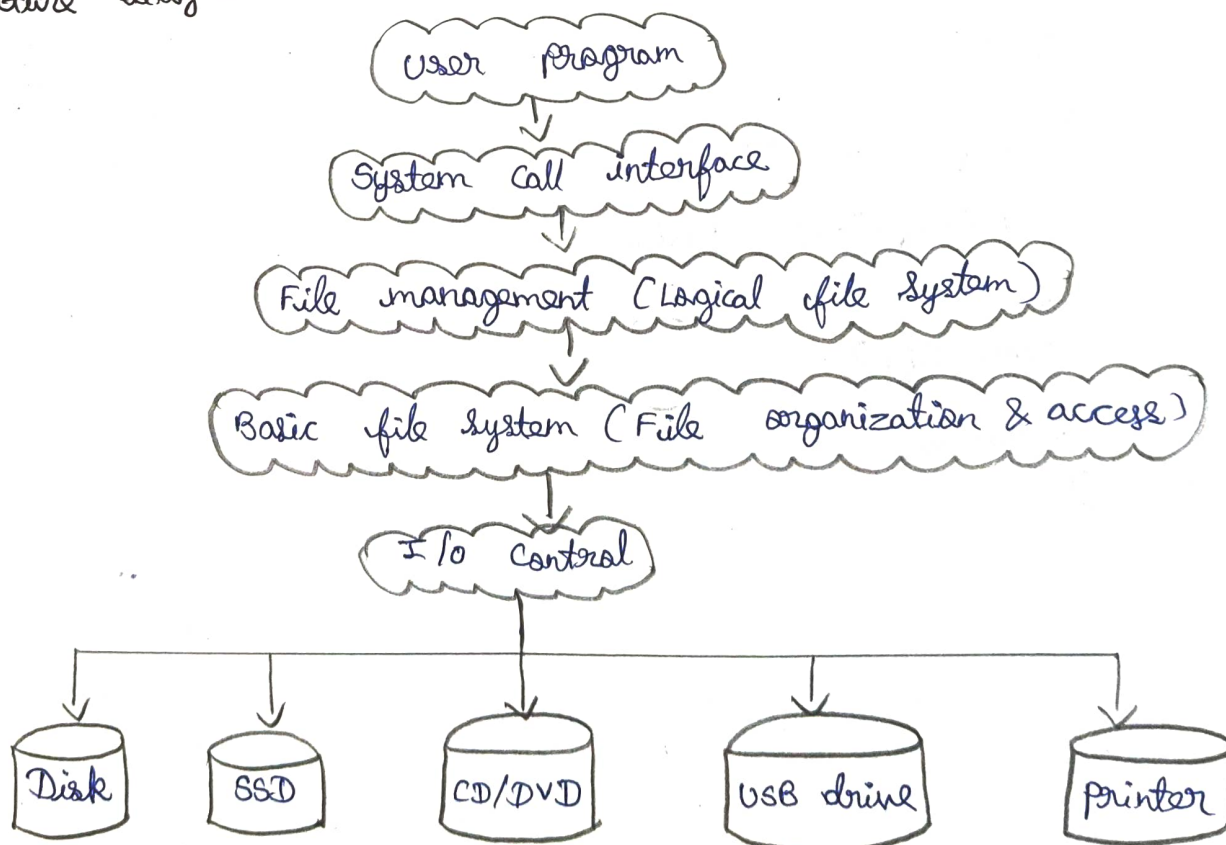
File management system calls

- 1) `open()` - opens a file
- 2) `read()` - Reads data from a file
- 3) `write()` - Writes data to a file
- 4) `close()` - closes a file
- 5) `lseek()` - Repositions the file pointer
- 6) `unlink()` - Deletes a file

Device management system calls

- 1) `open()` - opens a device
- 2) `read()` - Reads data from device
- 3) `write()` - Writes data to device.
- 4) `ioctl()` - performs device control
- 5) `close()` - closes the device

(b) Illustrate how the OS handles file operations using a layered structure diagram



c) Write a pseudo-code or C-based system call sequence to open, read and close a file.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    int fd, n;
```

```
    char buffer[100];
```

```
    fd = open("data.txt", O_RDONLY);
```

```
    if (fd == -1)
```

```
    {
```

```
        printf("Error opening file\n");
```

```
        return 1;
```

```
    }
```

```
    n = read(fd, buffer, sizeof(buffer) - 1);
```

```
    if (n > 0)
```

```
    {
```

```
        buffer[n] = '\0';
```

```
        printf("%s", buffer);
```

```
    }
```

```
    close(fd);
```

```
    return 0;
```

```
}
```